

Step 2 — A Real SQLite-Backed Store, and the Search Tool — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (the edge cases a real notes store has to survive). Read Basic first even if you already know SQL — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The pieces, and where each one lives](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The pieces, and where each one lives

Basic Version

Two separate problems live in this step, and it's worth keeping them apart in your head: (1) notes need to survive a restart, and (2) bad input (an empty note) needs to be rejected, loudly, before it ever reaches the database.

For persistence, Python's standard library already has everything you need — `sqlite3` — no extra package to install. A SQLite database is just a file on disk; `sqlite3.connect("notes.db")` opens it (creating it if it doesn't exist yet).

Things to use: - `sqlite3.connect(path)` — opens (or creates) the database file. - `CREATE TABLE IF NOT EXISTS notes (id INTEGER PRIMARY KEY AUTOINCREMENT, text TEXT NOT NULL, created_at TEXT NOT NULL)` — run once, safe to run every time the server starts since `IF NOT EXISTS` skips it if the table's already there. - `INSERT INTO notes (text, created_at) VALUES (?, ?)` — always use `?` placeholders, never paste the note text directly into the SQL string (the same "don't trust the input" lesson Doc06's tool-security section teaches, just with a database in place of a shell command). - `SELECT text FROM notes WHERE text LIKE ?` with a parameter like `f"%{query}%"` for a real substring search.

For rejecting empty input, `text.strip()` and a plain `if` check, raising your own exception class, is all you need — no library required.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The real shape is 3 small functions in `db.py` :

```
def init_db(path: str) -> None:
def insert_note(path: str, text: str) -> int:      # returns the new note's id
def find_notes(path: str, query: str) -> list[str]:
```

`init_db()` opens a connection, runs the `CREATE TABLE IF NOT EXISTS` statement, and commits — call it once, at server startup, before `mcp.run()` . Each of `insert_note()` and `find_notes()` should open its own short-lived connection, do its one query, and close it — SQLite handles many short connections from one process fine, and this avoids holding one connection open (and possibly stale) for the server's entire lifetime.

`created_at` should be a real timestamp — `datetime.now().isoformat()` is a plain, sortable string, good enough here without pulling in a new library.

For the validation error, define it once in `exceptions.py` :

```
class EmptyNoteError(Exception):
    pass
```

and raise it from inside `add_note()` in `server.py` — *before* calling `insert_note()` , not inside `db.py` . The database layer shouldn't need to know about the specific rule "empty text is invalid" — that's a decision about what a valid *note* looks like, which belongs with the tool, not with the storage code underneath it.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Two real edge cases worth thinking through before you call this step done.

First — what does the tool actually return when `EmptyNoteError` is raised? Doc06's "getting a tool's failure back to the model, correctly" section is the direct answer: don't let the exception propagate up and crash the server process. Catch it inside `add_note()` itself (or let FastMCP's own error handling turn a raised exception into an error result — check which behavior your installed SDK version actually gives you by testing it, since this is exactly the kind of detail worth verifying against real behavior instead of assuming), and make sure whatever the client receives clearly says *why* it failed ("note text cannot be empty"), not a bare stack trace.

Second — what happens to `search_notes` when the store is completely empty (server has never had a single successful `add_note`)? Your query should return an empty list cleanly, not raise anything — an empty result is a completely valid, expected outcome for a search, not a failure. Test this on purpose: call `search_notes` before adding anything, and confirm you get `[]` back, not an error.

A third thing worth noticing, even though it's not something you need to fix this step: **SQL injection**. Because you're using `?` placeholders everywhere (never f-string-ing the note text or query directly into a SQL statement), this server is already safe against it. This is the database version of Doc06's tool-security section — the model (or a malicious client) fully controls the `text` and `query` arguments, and parameterized queries are what stand between that untrusted input and your database actually running whatever string it contains as code.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate get the happy path working — real persistence, and empty input rejected. Advanced checks the two things a happy-path test won't catch on its own: that a rejected note produces a *clear* error the client can actually read, and that an empty database is handled as a normal case, not a bug — plus a reminder of exactly why the `?` placeholder habit from Intermediate isn't just style.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
db.py:
    init_db(path): create the notes table if it's not already there

    insert_note(path, text): insert one row, return its new id

    find_notes(path, query): select notes whose text contains query

exceptions.py:
    EmptyNoteError, a kind of Exception

server.py:
    add_note(text):
        if text (stripped) is empty: raise EmptyNoteError
        insert it, return a confirmation with its id

    search_notes(query):
        return the list of matching note texts
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
# db.py
import sqlite3
from datetime import datetime

def init_db(path: str) -> None:
    conn = sqlite3.connect(path)
    conn.execute(
        """CREATE TABLE IF NOT EXISTS notes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            text TEXT NOT NULL,
            created_at TEXT NOT NULL
        )"""
    )
    conn.commit()
    conn.close()

def insert_note(path: str, text: str) -> int:
    conn = sqlite3.connect(path)
    cursor = conn.execute(
        "INSERT INTO notes (text, created_at) VALUES (?, ?)",
        (text, datetime.now().isoformat()),
    )
    conn.commit()
    new_id = cursor.lastrowid
    conn.close()
    return new_id

def find_notes(path: str, query: str) -> list[str]:
    conn = sqlite3.connect(path)
    rows = conn.execute(
        "SELECT text FROM notes WHERE LOWER(text) LIKE ?",
        (f"%{query.lower()}%",),
    ).fetchall()
    conn.close()
    return [row[0] for row in rows]
```

Write `exceptions.py` and the `server.py` tool functions yourself from this, then compare against the [Solution](#).

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's the missing piece from Hint 1 — the validation and error handling wired into `server.py`, fill in the two tool bodies yourself:

```
# server.py (partial)
from exceptions import EmptyNoteError
from db import init_db, insert_note, find_notes

DB_PATH = config.notes_db_path
init_db(DB_PATH)

@mcp.tool()
def add_note(text: str) -> str:
    """Add a new note to the local notes store. Rejects empty or
    whitespace-only text with a clear error instead of storing it."""
    cleaned = text.strip()
    if not cleaned:
        # your turn: raise EmptyNoteError here, with a clear message,
        # and make sure server.py's error handling turns it into a
        # clean result the client can see -- not a crash
        ...
    new_id = insert_note(DB_PATH, cleaned)
    return f"Note added with id {new_id}."

@mcp.tool()
def search_notes(query: str) -> list[str]:
    """Search stored notes for a substring match, case-insensitive."""
    # your turn
    ...
```

Fill in both `...` sections yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same underlying idea (a table, an insert, a search) at 3 completeness levels — Basic only describes the plan, Intermediate is real, runnable `db.py` code, and Advanced wires that code into `server.py` with the validation check that's the actual point of this step, left for you to finish. Getting `EmptyNoteError` to turn into a clean, readable client-facing error (not a stack trace) is the one piece worth double-checking by hand before moving on.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)